

AID’N et GFD

Comparaison sur deux smokes

RAPPORT DES DEUX ESSAIS

Émis le 02/10/2026 à 18:59 EDT

Sur cette paire exploratoire, aucun gain de qualité du code avec AID’N n’est démontré. Les deux produits passent les 21 critères présélectionnés. Le contrôle complémentaire du titre contenant NUL échoue uniquement avec AID’N; le bras sans AID’N a découvert et corrigé ce défaut dans son budget. AID’N apporte une gouvernance et une traçabilité observables, au prix de 1.82 fois le temps et 4.72 fois les tokens natifs de développement. Cela ne préjuge pas de sa valeur sur un projet plus vaste ou une collaboration longue.

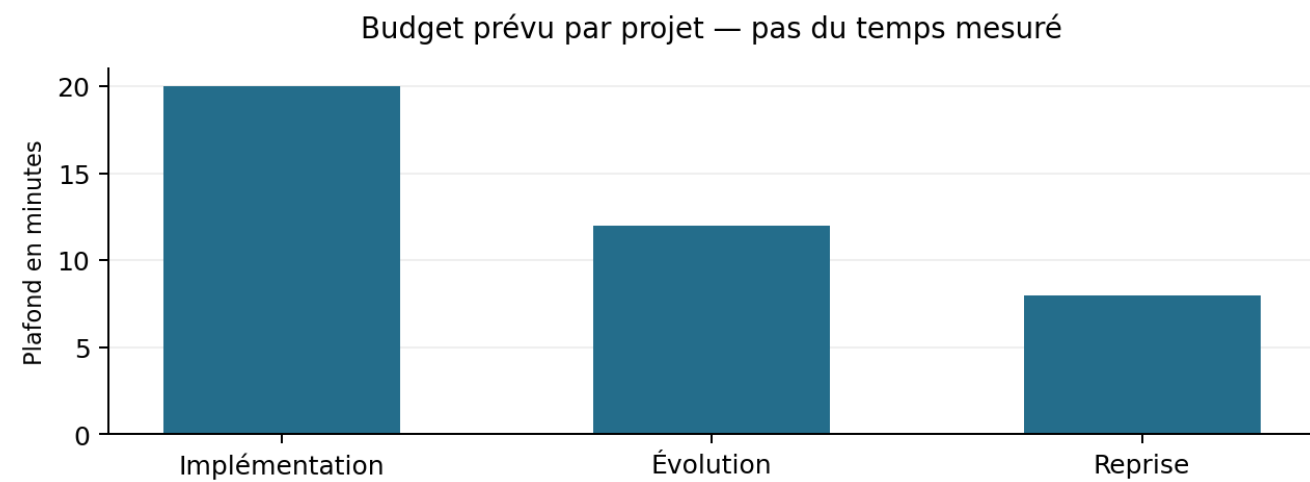
Question et comparaison

Sur le même sujet et avec GPT-6.1 medium, AID’N améliore-t-il le comportement et la qualité du code, et à quel coût de temps et de tokens ? La contribution de GFD sera analysée avec la limite d’attribution acceptée : deux bras ne permettent pas de l’isoler du reste d’AID’N.

Condition	Smoke A, exécuté en premier	Smoke B, exécuté ensuite
Workflow	AID’N du dernier dev, package épinglé	Sans assets ni runtime AID’N
Stockage AID’N	PostgreSQL 17.6 canonique; aucun fallback SQLite	Sans objet
Application	API de réservation, Node 22.13 / SQLite	Même sujet, starter et SQLite
Budget	40 min maximum; arrêt dur à 45 min	Même fenêtre de 30–45 min
Évaluation	Même évaluateur externe et grille de qualité	Même évaluateur externe et grille de qualité

Sujet et déroulement

API HTTP de réservation de salles : validation, dates explicites, conflits temporels, idempotence, requêtes concurrentes, persistance et migrations. Une évolution ajoute l’annulation sans perdre l’historique. La dernière phase reprend le dépôt dans une nouvelle conversation.

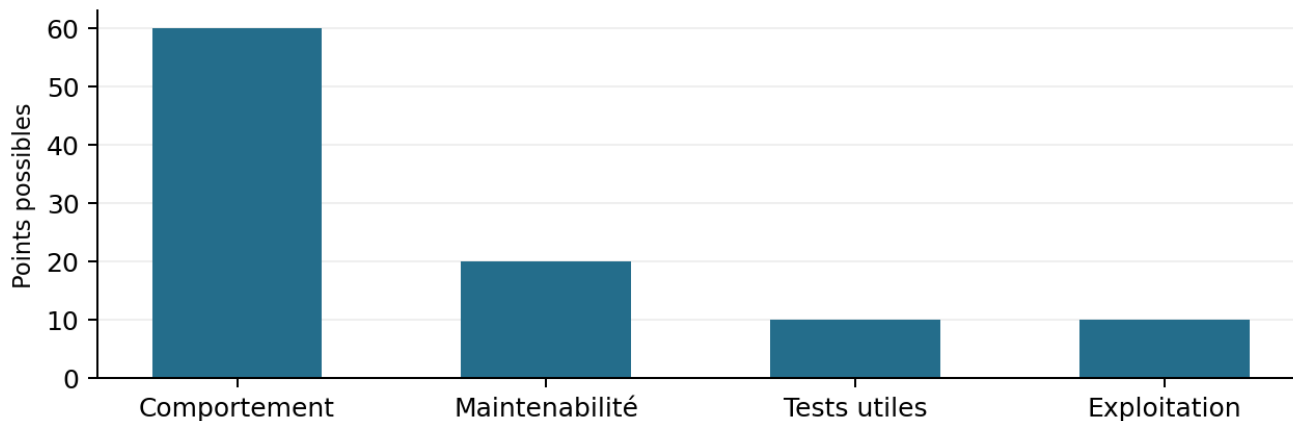


Préparation et installation AID’N/PostgreSQL hors budgets. Leur durée complète et les tokens de l’expérimentateur n’ont pas été mesurés exhaustivement : coût total de première utilisation UNAVAILABLE. Un préflight natif réussi fournit 12 707 tokens partagés, affichés séparément et non attribués à un bras. Les tentatives échouées sans compteur final restent UNAVAILABLE.

Qualité : résultat métier avant conformité documentaire

L'évaluateur externe exerce 21 critères : HTTP, erreurs, dates, chevauchements, limites, salles indépendantes, rejouabilité, conflits de clés, concurrence, redémarrage, annulation et conservation d'une base de phase 1. Il ne se contente pas de tests écrits par l'agent. Un défaut critique reste visible même si la documentation est bonne.

Grille figée avant les essais — pas des notes obtenues



La revue qualitative utilise des exports X/Y sans les assets AID'N. Son auteur, les preuves et les limites du masquage sont documentés. Le bras sans AID'N n'est pas pénalisé pour ne pas employer ses outils ou son vocabulaire.

Tokens : compteurs natifs et efficacité

Mesure	Calcul ou règle
Total	Entrée + sortie. Aucun ajout séparé du cache ou du raisonnement.
Entrée neuve	Entrée moins entrée servie depuis le cache.
Sortie	Afficher le raisonnement comme sous-ensemble de la sortie.
Taux de cache	Entrée servie depuis le cache / entrée totale.
Efficacité	Tokens par critère validé et par point de qualité, avec dénominateurs.
Reprises	Dernier total cumulé par thread; différences entre snapshots, sans double comptage.
Autres coûts	Cache écrit et éventuels appels auxiliaires séparés; valeurs absentes = UNAVAILABLE.

La source officielle de Codex 0.159.0-alpha.3 confirme que `turn.completed.usage` reprend le dernier total du thread. Les JSONL, erreurs et identités du modèle sont conservés. Les préflights échoués ne fournissent pas de compteur final : absence de données n'est pas zéro token. Pas de montant en euros inventé pour un abonnement ChatGPT.

GFD : périmètre réellement observé

Relier obligations, responsabilités, décisions, exceptions et preuves aux commandes et artefacts réellement produits. Distinguer mécanisme présent, mécanisme utilisé et résultat obtenu. Ne pas attribuer exactement à GFD les tokens d'un tour mêlant gouvernance et code.

Le diagnostic initial indique une adoption partielle effective dans le package source, et une adoption cliente absente. L'installation ne transmet pas automatiquement cette acceptation au projet. Une déclaration ne prouve ni conformité ni exécution. Le rapport conservera ces distinctions.

Identité et état des essais

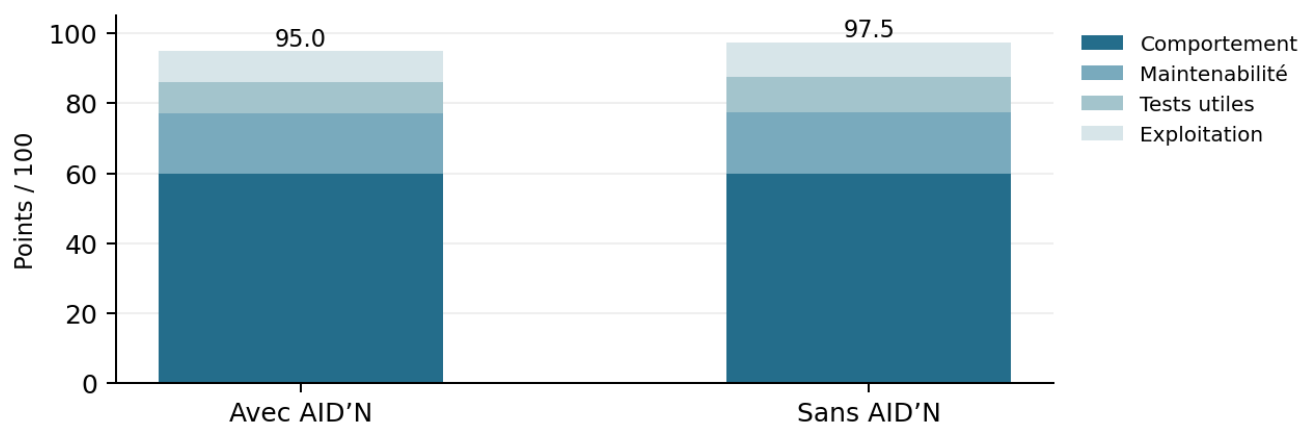
Vérification	État
Dernier dev observé	89a709b9ccc3a1effb91356c654cb9fc3b5e4650
Version / fichiers installés	0.12.0 / 968 fichiers comparés au commit : PASS
PostgreSQL AID'N	17.6; schéma 3, 20 tables; admission canonique : PASS
Instrumentation	6 tests de compteurs et contrôle négatif de l'évaluateur : PASS
Deux développements	EXÉCUTÉS — preuves et scores ci-après
Qualité et tokens A/B	Résultats mesurés et limites ci-après
Accès natif	Connexion native réussie; deux bras et leurs nouveaux threads terminés
Modèle demandé	gpt-6.1-sol / medium confirmé par thread/start; aucun reroutage observé
Revue des hooks	Revue humaine explicite, contrôles TUI officiels, hooks/list : 2 enabled/trusted; fichiers inchangés

Portée de la conclusion finale

Une paire, N=1 par bras : résultat exploratoire sur un sujet et une fenêtre. Aucun test de significativité ni effet causal général établi. L'ordre A puis B et le cache restent des biais possibles. Les échecs seront conservés; aucune correction des produits par l'expérimentateur après le temps alloué.

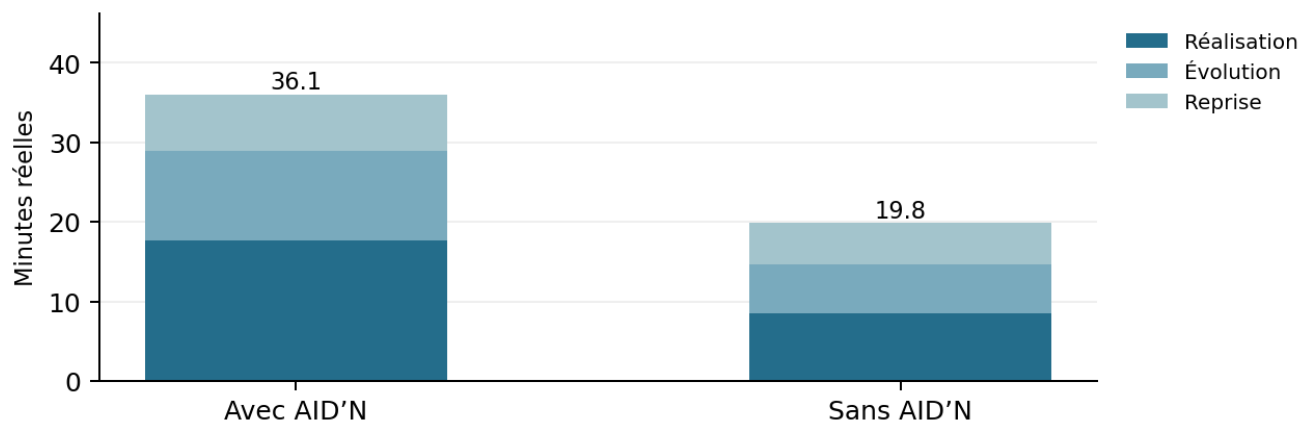
Comparaison observée

Qualité observée, N=1 par bras



Mesure	Avec AID'N	Sans AID'N
Comportement	60.00	60.00
Maintenabilité	17.00	17.50
Tests utiles	9.00	10.00
Exploitation	9.00	10.00
Critères externes validés	21 / 21	21 / 21
Temps de développement	36.07 min	19.84 min

Temps mesuré par phase

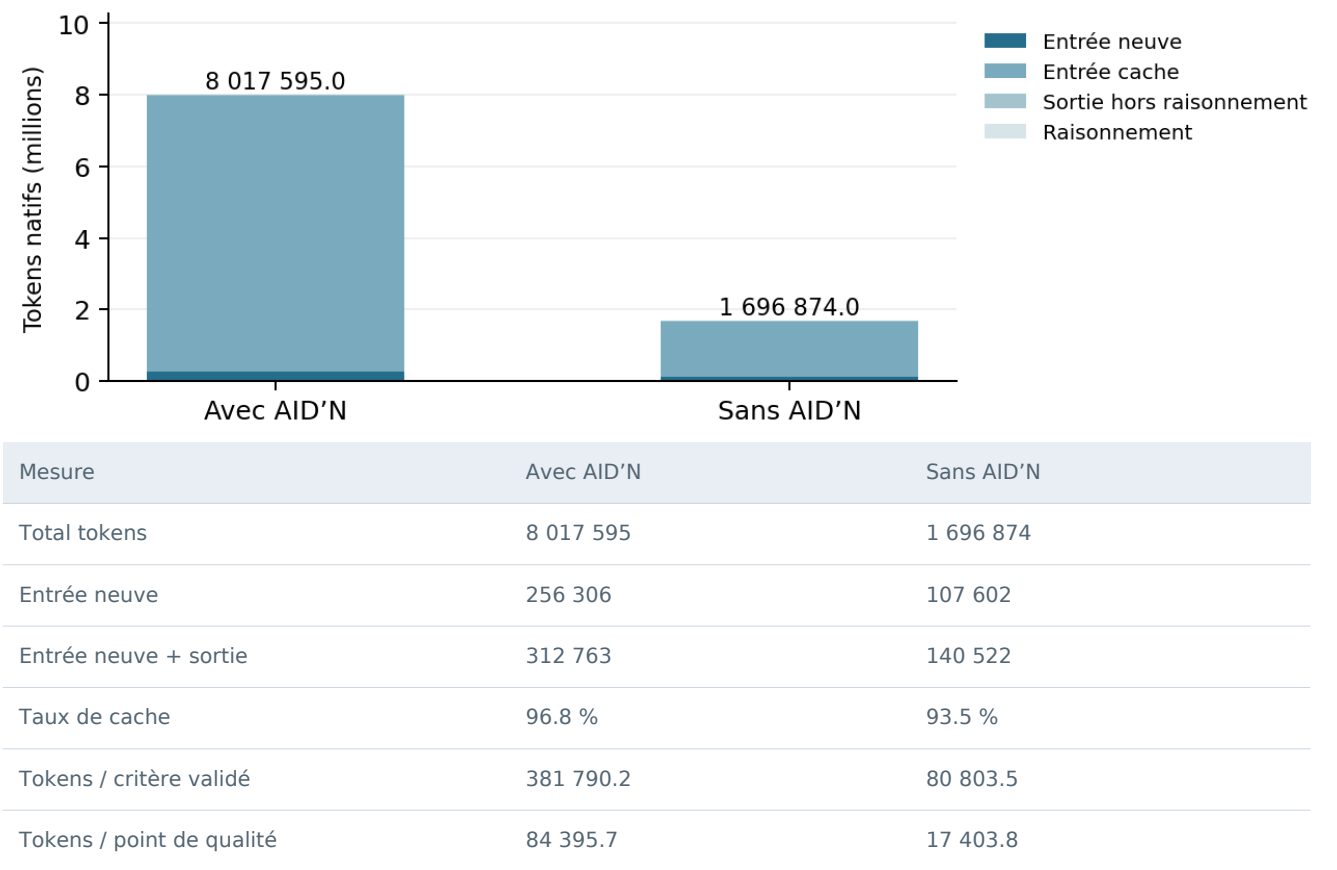


AID'N : 36.07 min; sans AID'N : 19.84 min. Même plafond de 40 min, phases de 20/12/8 min et arrêt dur à 45 min. Le bras sans AID'N a terminé en moins de 30 min; aucune attente artificielle n'a été ajoutée. AID'N : 120 commandes natives, 131 hooks (7,30 s de durée cumulée), dont un refus effectif d'édition pour état/scope invalides. Sans AID'N : 49 commandes, aucun hook. Chaque bras a 9 commandes affichées comme failed; ce nombre mêle erreurs et refus et ne mesure pas seul des défauts. Les métadonnées, reanchors, branches, audits et clôtures expliquent une friction observée, sans attribuer tout le délai à GFD. Des contraintes du sandbox et le shell démarrant en Node 24 influencent également le coût.

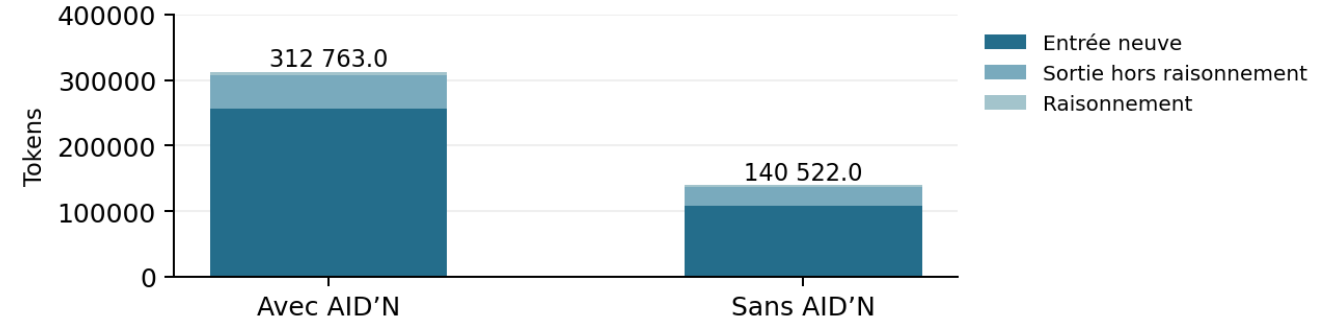
Tokens et efficacité observés

Périmètre : compteurs natifs des deux conversations du développeur GPT-6.1-Sol medium, par projet. Les éventuels modèles de revue automatique ne sont pas isolés par cette instrumentation; leurs compteurs séparés sont UNAVAILABLE. Le travail de préparation de l'expérimentateur est également hors de ces totaux. Cette limite empêche de conclure à une économie globale de tokens.

Décomposition sans double comptage



Entrée neuve + sortie — volume hors entrée cache



Totals natifs AID'N / sans : 8,017,595 / 1,696,874. Cache entrée : 96.78 % / 93.53 %. Entrée neuve + sortie : 312,763 / 140,522, soit 2.23 fois. Ce dernier volume n'est pas une facture. Le cache et le raisonnement sont des sous-ensembles; jamais ajoutés une deuxième fois. Dernier total du thread partagé phases 1-2 plus total du nouveau thread phase 3. Les snapshots JSON-RPC natifs complets sont conservés; native-usage.jsonl est leur normalisation pour le calcul. Coûts des modèles de revue automatique non isolés; aucune estimation en euros. Ordre fixe A puis B et cache partiellement partagé limitent l'attribution.

Revue et défauts observés

Revue sur exports X/Y sans assets AID’N, par le même observateur ayant piloté les essais : anonymisation documentaire, pas double aveugle indépendant. Les notes 95/100 et 97,5/100 incluent 40 points de jugement qualitatif; leur petit écart ne constitue pas une preuve statistique. Le défaut de titre est plus informatif que ces décimales. Aucun double booking, perte de réservation ou échec de migration n’a été observé par l’oracle commun.

Maintenabilité, 17/20 et 17,5/20

Les deux produits séparent HTTP et persistance, utilisent des paramètres SQL, des transactions et des triggers. AID’N : app.js regroupe validation et routage; store.js contient migration et stockage, et fournit `HttpError` au HTTP. Sans : validation.mjs séparé, database.mjs plus long, boilerplate transactionnel répété. Lire les mappings et justifications détaillés dans `quality-review.json`. Le bras sans dispose d’un `retry` borné pour la création WAL et d’une conversion explicite des octets du titre; ce ne sont pas des points gagnés pour du vocabulaire de gouvernance.

Tests utiles, 9/10 et 10/10

Huit tests AID’N et quinze sans passent dans la vérification indépendante finale Node 22.13.0. Les deux suites utilisent des bases réelles, injections d’échec, migrations, concurrence entre processus et redémarrage. Le bras sans ajoute arrêts forcés, contention de locks, annulation rollback et régression NUL. Les nombres ne sont pas le barème : c’est la couverture utile et réellement exécutée qui est jugée.

Exploitation, 9/10 et 10/10

Les deux README documentent configuration, migration, sauvegarde après arrêt des writers, rollback par restauration et limitations. Sans : arrêt borné à cinq secondes et handoff sur Node 22. AID’N : arrêt gracieux sans borne forcée; la reprise a validé sur Node 24 en le déclarant honnêtement. Le contrôle final commun sur Node 22 réussit. Pas de downgrade ni certification de production dans aucun bras.

Défaut reproductible hors grille : conservation du texte

Même requête dans les deux produits sur Node 22.13.0 : `title = Before<caractère NUL>After`. Avec AID’N, création et jeu conservent le titre, mais GET et DELETE retournent Before. Sans AID’N, les quatre réponses conservent le titre intégral. Cause visible : `store.js:65-81` lit directement le TEXT du driver Node 22; `database.mjs:125-129` dans le bras sans lit les octets et les décode. Aucun produit n’a été réparé après budget. Le cas a été ajouté après découverte par le bras sans et appliqué aux deux : résultat complémentaire, pas modification rétroactive des 60 points figés.

Contribution GFD et limites

Le diagnostic initial et final déclare l'adoption partielle effective du package source, mais l'adoption cliente GFD reste absente, sans héritage automatique. Conformité et certification GFD ne sont pas évaluées. Avec AID'N, session/cycles S001-C001-C002, DoR, matrice d'usage, périmètre gelé, blocage natif d'une édition et artefacts PostgreSQL sont réellement observés. Le handoff a d'abord été bloqué sur la branche du cycle clos, puis a passé après fast-forward vers la branche de session; la reprise a rafraîchi des ancres incohérentes. Cette traçabilité n'a pas évité le défaut NUL. Les hooks d'édition couvrent apply_patch/Edit/Write, pas une validation sémantique universelle de toutes les commandes shell. Avec deux bras, pas d'effet causal GFD isolé, pas de tokens précisément attribuables à GFD. Recommandation : simplifier l'initialisation, rendre explicites les transitions de clôture/handoff et réduire le contexte répété, en conservant les admissions utiles; ne pas généraliser à la gouvernance d'un gros projet.

Corrections du harnais et limites de l'oracle

Le lanceur initial supposait src/server.mjs alors que le contrat public demande npm start. Corrigé pour les deux produits sans modifier les critères; version précédente et six tests du harnais conservés. La base A de phase 1 a été capturée pendant la phase 1, après implémentation; B à sa clôture. Les hashes du code et données d'origine sont conservés, et les deux migrations passent. Le binaire Node téléchargé par B est vérifié avec le checksum officiel et identique au binaire partagé. L'oracle ne couvre pas exhaustivement l'Unicode, tous les ISO 8601, les pannes d'alimentation ou la charge prolongée. Le cas NUL est post-hoc et marqué comme tel. N=1 par bras, ordre A puis B, même observateur, absence de troisième bras GFD : aucune causalité générale ni significativité statistique.

Résultats comportementaux, critère par critère

Critère	Avec AID'N	Sans AID'N
health	PASS	PASS
room_creation	PASS	PASS
reservation_creation	PASS	PASS
reservation_lookup	PASS	PASS
unknown_resources	PASS	PASS
input_validation	PASS	PASS
timestamp_validation	PASS	PASS
overlap_rejected	PASS	PASS
adjacent_allowed	PASS	PASS
rooms_independent	PASS	PASS
idempotency_replay	PASS	PASS
idempotency_conflict	PASS	PASS
concurrent_collisions	PASS	PASS
concurrent_idempotency	PASS	PASS
restart_persistence	PASS	PASS
cancellation_history	PASS	PASS
cancel_frees_slot	PASS	PASS
cancelled_creation_replay	PASS	PASS
http_errors	PASS	PASS
sql_input_safety	PASS	PASS
phase1_upgrade_preservation	PASS	PASS

PASS signifie que le comportement attendu a été observé par cet oracle, avec les entrées choisies. Il ne constitue pas une preuve exhaustive pour toutes les dates ISO, charges, crashes ou conditions de déploiement. Les cas non exécutés restent distincts des échecs.

Sources et preuves

Dossier local : /workspace/aidn-smoke-ab ; PROTOCOL.md, REVIEW.md, evidence/, package/manifest.json. SHA-256 du tarball : 5cec528f1a7fe4afc5a10dec1be94750165c84f738fc23a2fab1692563d9236

Sources officielles, commit 3b01b36fa5eb96ba82a776bd3c2fc57f8969181f :

https://github.com/openai/codex/blob/3b01b36fa5eb96ba82a776bd3c2fc57f8969181f/codex-rs/exec/src/exec_events.rs

https://github.com/openai/codex/blob/3b01b36fa5eb96ba82a776bd3c2fc57f8969181f/codex-rs/exec/src/event_processor_with_jsonl_output.rs

AID'N : docs/CODEX_NATIVE_QUALIFICATION.md, étape 4; diagnostic d'adoption et manifeste du package.